백예지 기술이력서

회사경력

재직기간	회사명
2022.03 ~ 재직중	㈜ 미디어로그

경험기술

- Java, Spring Boot 3.x, Spring WebFlux, Spring Security, RESTful API
- JPA, Querydsl, Mybatis, Oracle, MySQL, PostgreSQL
- ElasticSearch, Apache Hadoop, Impala
- OAuth2, OpenID Connect(OIDC), Keycloak, Okta
- Linux, Apache Tomcat, GitLab, TeamCity
- Vue.js, css3, HTML5, Yarn, Vite, Tailwind
- (AI도구) Kiro, Claude Code, Gemini

업무역량

Spring Security 기반의 OIDC 표준 로그아웃 핸들러를 확장하여 사내 세션과 WAS 세션을 안전하게 동시 파기하는 인증/인가 아키텍처를 표준화하고, 사내 SSO(OIDC) 연동 과정에서 폐쇄망 인프라의 제약 조건과 Nginx 프록시 환경의 인증 유실 문제를 해결하였습니다.
또한, 다중 AI 엔진 운영 관리 시스템 구축 시, FE와 API 스펙 정의를 리딩하고 일정을 관리하여 협업 병목을 해소했습니다. BE 영역에서는 도메인 중심(Domain-Driven Design)의 아키텍처 설계를 진행하여 엔진별 비즈니스 로직과 다형성 구조를 명확히 정의하고, Single Table 구조의 DB 한계를 극복하기 위해 도메인-영속성 레이어 분리 구조를 도입해 코드의 결합도를 낮췄습니다.
나아가 MapStruct 기반 공통 레이어 구축과 Elasticsearch Aggregation 연동을 주도하여 실시간 AI 이용 통계 파이프라인의 처리 성능을 극대화했습니다.

자격증

취득일자	자격증명	발급기관
2025.11	SQLD	한국데이터산업진흥원
2024.12	AWS Cloud Practitioner	AWS

업무경험

㈜ 미디어로그 (2022.03 ~ 재직중)

[수행프로젝트]

통신사 품질 및 고객경험(CEM) 데이터 분석 플랫폼 구축

[해결방안 및 사용기술]

1. 대용량 위치 정보(GIS) 및 민감 정보 암호화 기반 지도 연동 API 설계

- 주생활지 데이터 기반의 위경도 좌표 데이터를 추출 및 가공하는 위치 정보 조회 쿼리 설계 및 API 구현
- **지도 연동 모듈 파라미터 설계와 민감 정보 암호화 및 마스킹 처리**
- 사내 공통 지도 연동 모듈(카카오맵 iframe)의 request 스펙에 맞춰 전사 공통 Crypto 유틸을 활용해 암호화된 위경도와 조회자ID, IMSI/MSISDN 값과 지도 동작 타입(Move/Refresh/Open)을 전달하도록 연동 로직 구현
- Move/Refresh: 조회 시점 위치에 핀(Pin) 표시 / Open: 조회자가 마지막으로 조회한 지역만 표시
- 화면에 노출될 개인정보를 전사 정보보호규정 내 정보보안지침(이름, 주민번호 등)의 가이드대로 마스킹
- maskingUtil 클래스를 생성해 마스킹 유형별(이름, RRN, 카드, 전화번호, 계좌번호 등) 규칙을 Enum으로 관리하여 향후 마스킹 항목이 추가될 경우 Enum 상수와 마스킹 로직만 추가하면 되도록 유지보수성과 확장성을 고려하여 설계함

2. Apache Impala(Hadoop) 기반 대용량 통신 트래픽 데이터 쿼리 튜닝 및 성능 최적화

- 통화, 인터넷 사용 기록, 장애 발생 등 다량의 데이터가 조회되거나, 기간을 넓게 설정한 경우, 대용량 데이터를 받아오지 못하거나 리소스(CPU/메모리)가 부족하여 쿼리 타임아웃이 발생
- 전체 조회 쿼리를 **초단위 데이터의 분단위로 1차 파티셔닝**
- **전화번호 끝자리와 위경도 값으로 2차 파티셔닝하여 스캔 대상 데이터를 축소**하여 조회 성능 개선
- 이용률이 높은 기지국의 위경도를 기준으로 주생활지를 산출하여(예: 용산 기지국, 마곡 기지국) **2차 파티션 키로 설계**
- **Impala 쿼리 및 spring 서버 양쪽에 타임아웃을 5분으로 설정**해 장시간 실행되는 쿼리를 강제 종료
- 운영팀에 **Impala 리소스(CPU/메모리) 증량을 요청**하여 근본적인 리소스 부족 문제를 해결
- 중복 조회로 인한 서버 부하 방지를 위해 UI에서 버튼을 disable 처리하고 서버에서 (동일 IP 중복 요청 차단) 이중 처리 로직을 구현

3. 빅데이터 플랫폼(Impala) 보안 강화를 위한 Kerberos 인증 체계 제안 및 Spring boot 연동 (진행중)

- 기존 LDAP 기반의 **평문/단방향 인증 방식은 보안에 취약할 수 있음을 인지** 후, 인프라 운영팀에 **Kerberos 인증 체계 전환을 제안**
- 보안 프로토콜 변경에 따른 Spring Boot, Impala 커넥션 및 권한 검증 아키텍처 재설계
- Spring Boot에서 Keytab 및 SPN 기반 인증 자격 증명을 보안 프로퍼티와 연동하여 동적 관리 설계
- 10~24시간의 커버로스 티켓 만료주기로 인한 장애를 방지하기 위해 Hadoop 오픈소스 API 의
- UserGroupInformation을 사용해 Kerberos 티켓(TGT) 자동 갱신 프로세스 적용

[수행프로젝트]

사내 SSO 연동 및 OIDC 기반 인증/인가 아키텍처 표준화

[해결방안 및 사용기술]

1. 사내망 분리 및 아웃바운드 차단 정책으로 인한 OIDC 엔드포인트 수동 매핑

- issuer-uri 기반의 자동 구성을 대신하여, OAuth2 속성 계층 구조를 활용해 registration 영역과 provider 영역을 분리하여 yml 파일에 수동 매핑
- registration 노드에 클라이언트 ID와 secret 및 grant type을 정의
- provider 노드에 **인가(Authorization), 토큰, 사용자 정보(user-info), jwk set의 endpoint 정의**

2. 기존 폼 로그인 방식에서 사내 도메인 세션 기반 자동 로그인 전환 및 인증 예외 처리 표준화

- 기존 Keycloak 기반 폼 로그인 방식에서, 이미 로그인된 세션을 보유한 상태의 서비스 페이지 접속 시 별도 절차 없이 자동 인증되는 구조로 전환함
- DefaultOidcUser를 상속하여 **사내 SSO 규격에 맞게 커스텀 UserPrincipal(SessionUser)를 설계해 Spring Security Context에 통합**함
- 공통 SSO Claim에 해당 서비스의 세부 권한 정보가 담겨 있지 않아 **OAuth2UserService.loadUser() 내부에 사용자 DB 매칭 로직을 구현**하여, **SSO 인증 완료 후 자체 DB 기준의 해당 시스템 권한(Role)을 2차 검증하는 보안 로직** 추가
- 인증 성공/실패 시점의 HTTP 흐름을 제어하기 위해 AuthenticationSuccessHandler 및 AuthenticationFailureHandler를 구현
- 프록시(Nginx/로드밸런서) 환경을 고려하여 X-Forwarded-For 헤더 및 User-Agent 분석을 통해 클라이언트의 실제 IP와 OS/브라우저 정보를 정확히 추출하는 로직을 구현
- @EventListener와 @Async 기반의 **비동기 이벤트 핸들링 구조로 설계**하여, 인증 성공/실패 시 로그테이블에 적재할 때 메인 스레드의 지연을 방지
- 로그테이블 적재 시 상세 정보를 위해 Nginx/로드밸런서 환경에서 X-Forwarded-For 헤더를 파싱해 실제 IP를 파악하고, User-Agent로 OS/브라우저 정보를 추출하는 유틸리티 클래스를 구현

3. Spring Security 및 OIDC 기반의 사내 SSO 연동 로그아웃 프로세스 구축

- SecurityFilterChain 내 설정을 통해 WAS 내부 세션 무효화(.invalidateHttpSession(true))
- 인증 객체 제거(clearAuthentication(true)), JSESSIONID 쿠키 삭제로 로컬 인증 정보를 완전 파기
- OidcClientInitiatedLogoutSuccessHandler를 상속받은 커스텀 로그아웃 핸들러를 구현
- 로컬 세션 만료 후 사내 SSO의 로그아웃 endpoint로 redirect

4. 로컬과 배포 환경 간 CORS 및 세션 미공유 이슈

- 프론트엔드와 백엔드 간 도메인 불일치로 인한 세션 쿠키 미공유 문제를 해결하기 위해 Nginx를 리버스 프록시로 구성하여 동일 origin 환경을 구축
- /oauth2/*, /api/* 등 주요 인증 API 경로가 WAS로 매핑되도록 Nginx Upstream 라우팅 설정으로 CORS 블로킹과 세션 공유 제약을 해결
- Nginx 단에서 X-Forwarded-Proto, X-Forwarded-For 등 필수 프록시 헤더 포워딩 설정하여 프록시 환경에서 사내 SSO 리다이렉트 도메인 및 HTTPS 정보가 유실되는 문제 해결

- Spring Boot 환경 설정(server.forward-headers-strategy: framework)으로 Nginx가 전달한 프록시 헤더를 Spring Security가 신뢰하도록 연동

[수행프로젝트]

AI 통합 플랫폼 채팅 서비스 고도화

[해결방안 및 사용기술]

1. 공통 엔드포인트 구축 및 리액티브 파이프라인 단일화

- 신규 AI 엔진이 추가될 때마다 개별 컨트롤러를 새로 생성해야 했던 비효율적인 구조를 개선하고, 다중 엔진(ixi, ChatGPT, Gemini 등)을 수용하는 단일 공통 endpoint로 통합
- @RequestMapping 설정을 통해 JSON(/chat)과 실시간 SSE(/chat/stream) 요청을 공통으로 수용하고, 리턴 타입을 Mono<ResponseEntity<Flux<String>>>로 정의하여 공통 리액티브 파이프라인으로 구축
- 엔진별 연동 규격을 표준화하기 위해 공통 인터페이스(aiApiClient)를 설계하고, 각 엔진이 이를 상속하여 독립 구현하도록 구조화해 기존 코드 영향도를 최소화하고 신규 엔진 확장 편의성 개선

2. 리액티브 스트림 제어 최적화 및 예외 처리 표준화

- 실시간 스트리밍(SSE) 데이터를 화면에 즉시 보여주면서, API에서는 AI Client의 응답 원본을 로그 테이블에 적재하기 위해 **chunk 데이터를 최종 답변 형태로 조립하는 병합 로직을 구현**
- AI의 답변이 오래 걸려 연결이 끊어지는 현상을 보완하기 위해 **Keep-Alive(heartbeat) 로직을 넣고, 타임아웃 설정으로 SSE 스트리밍 연결의 안정성**을 개선
- doOnCancel을 활용해 '응답 대기 중 사용자의 취소'와 '스트리밍 중 취소' 시점을 세분화해 로그에 남겨 검증 데이터를 확보하고
  doFinally 단계에서 원본 및 파싱 응답을 Elasticsearch에 비동기 적재하도록 연동
- AI 엔진 자체 에러(4xx/5xx)와 네트워크 에러를 doOnError로 구분하고, onErrorResume을 통해 표준화된 에러 포맷으로 변환하며 화면에 표출되도록 WAS 전반의 예외 처리 구조를 일원화

[수행프로젝트]

AI 챗봇 서비스 통합 운영 관리자 페이지 및 보안 강화 시스템 구축

[해결방안 및 사용기술]

1. 대용량 로그 통계 산출을 위한 ElasticSearch 복합 집계 쿼리

- 대규모 질문 이력 데이터의 통계 집계 성능 최적화 및 대시보드 연동을 위해 Spring Data ElasticSearch(3.5.5) 기반의 Aggregation 아키텍처를 도입
- 복잡한 집계 쿼리를 제어하기 위해 **NativeQueryBuilder를 직접 활용하여 구현**
- Bool Query를 활용해 권한, 공휴일 여부 등 다중 조건을 조합하여 1차 데이터 필터링 조회 로직 구현
- 시간대별/엔진별/사용자별 통계를 단일 쿼리로 처리하기 위해 Date Histogram -> Terms -> Cardinality 구조의 **Nested Aggregation을 설계**

- Date Histogram Aggregation(fixed_interval/calendar_interval)을 활용해 일별/월별 시간 단위 통계 생성
- Terms Aggregation을 활용해 AI 서비스(엔진)별 통계를 그룹화하여 집계
- 고유한 사용자 id 필드를 대상으로 Cardinality Aggregation을 최하위 버킷에 적용해 중복을 제거한 순 사용자 수 산출
- 전체 질문 수 집계 시, 별도 집계 쿼리 없이 반환되는 Total Hits 값을 그대로 활용하여 불필요한 연산을 제거
- 복잡한 집계 결과 값을 화면 통계 그래프 스펙에 맞춰 DTO 구조로 변환하여 화면에 반환

2. DDD 기반 도메인 모델링 및 JPA 상속 관계 매핑

- 여러 AI 엔진(Ixi, GPT, Gemini, 자사 AI 등) 정보를 체계적으로 관리하기 위해 객체지향적 도메인 주도 설계(DDD) 적용
- 각 엔진별로 다른 정보들을 하나의 도메인으로 통합하고자, JPA 단일 테이블 전략을 설계함
- @Inheritance(strategy = InheritanceType.SINGLE_TABLE)과 @DiscriminatorColumn(name = "API_TP") 설정을 통해 엔진별 공통 스펙을 정의한 부모 클래스를 생성
- 자식 클래스는 @DiscriminatorValue로 API_TP 컬럼에 들어갈 고유 값을 지정
- 신규 AI 엔진 규격이 추가되더라도 기존 스키마 변경 없이 대응 가능한 백엔드 구조를 구축

3. Burp Suite 모의해킹 기반 취약점 조치 및 서버 보안성 강화

- 애플리케이션의 보안성을 검증하고 사내 보안 가이드라인을 충족하고자, 웹 취약점 진단 도구 Burp Suite를 활용해 모의해킹 수행
- Burp Suite의 HTTP 요청/응답 패킷 인터셉트 기능을 활용해 로그인 응답값의 권한(role) 필드를 임의 변조해 관리자 페이지 접속 가능 취약점 발견
- 클라이언트 단의 값만을 신뢰하지 않도록 권한 검증 구조를 변경하고, 모든 API 요청마다 서버사이드(Session/SecurityContext)의 권한을 기준으로 재검증하도록 로직 강화
- 정상적인 메뉴 경로를 거치지 않고 URL을 직접 입력하여 우회하려는 취약점 발견
- Spring Security 필터 체인의 .requestMatchers().hasRole()을 활용해 세션 기반의 권한 체계를 구축하고, 정상적인 라우팅 경로를 거치지 않은 비인가 URL 우회 접근 시도를 서블릿 단에서 원천 차단함
- 화면상 마스킹 처리된 사용자 정보가 실제 HTTP Response에는 원본 데이터 그대로 노출되어 전송되는 취약점 발견
- MapStruct 매퍼에 별도 분리한 마스킹 클래스를 uses 속성으로 지정하고, 각 필드에 @Mapping의 qualifiedByName를 연결하여 @Named로 정의된 로직이 DTO 전환 시점에 자동 마스킹 되도록 구현

4. 아키텍처 표준화 및 PL 역할 수행

- 화면 설계서를 기반으로 API를 설계하여 프론트엔드 개발자와 협업 프로세스를 주도하여 전체적인 개발 공수를 단축함
- Git 브랜치 전략(feature/fix 접두어 및 배포 환경별 구조) 수립 및 승인 조건(최소 1인 이상 승인) 의무화를 통해 코드 품질 관리 프로세스 구축

[수행프로젝트]

CI/CD 자동화 구축

[해결방안 및 사용기술]

1. VCS 연동 기반 수동 배포의 TeamCity 파이프라인 자동화

- 개발자가 서버에 직접 접근해 Jar 및 웹 빌드 파일을 수동으로 업로드하던 기존 배포 방식의 에러를 해결하고자, TeamCity 기반의 CI/CD 자동 배포 파이프라인 구축
- Git 레포지토리 내 애플리케이션 소스 코드뿐만 아니라 application.yml 등 환경 설정 파일까지 VCS Trigger 감지 대상에 포함 (Branch filter 설정에 +:.**)
- 설정 파일 변경 커밋 시에도 파이프라인이 즉시 작동하여 서버로 자동 배포되도록 구현함으로써, 수동 반영으로 인해 발생하던 설정 유실 및 배포 실수를 원천 차단

2. Git 브랜치 전략 별 환경별 배포 프로세스 구분

- 개발 환경(Pre-production)과 운영 환경(Production)의 배포 수준을 다르게 설정하여 배포 효율성과 운영 안정성을 동시에 확보
- Git 브랜치(pre-production/production)와 연동하여 pre-production은 Git에 푸시되거나 풀 리퀘스트가 생성될 때 즉시 자동 배포되도록 Trigger를 추가하여 구현하고, production은 관리자 승인 단계를 거치도록 설계

3. Build Chain 및 Matrix Build 기반 N개 서버 통합 배포 및 구조 설계

- 멀티 서버 환경에서 서버 대수에 비례해 개별 배포 스텝을 늘릴 경우, 불필요한 컨피규레이션 낭비와 수동 관리에 따른 설정 누락 및 배포 에러 위험이 극대화됨
- Matrix Build 설정을 도입하여 다중 환경 설정을 하나로 일원화하고 파이프라인 관리 효율을 극대화함
- Snapshot Dependency를 구성하여 파이프라인 실행 중 신규 커밋이 생겨도 동일한 Git Revision을 참조하도록 소스 버전을 고정하고, 빌드-배포 간 순서를 보장하는 Build Chain을 구축하여 전체 파이프라인 흐름을 시각화
- 앞선 빌드 단계에서 검증이 완료된 Artifact(jar, asset 등)를 배포 컨피규레이션으로 전달 및 재사용하도록 Artifact Dependency 구성하여, 배포 파일의 구조적 무결성 확보
- 이를 통해 단 1회의 통합 트리거로 전 서버 일괄 배포가 완료되도록 파이프라인 단계를 축소 및 단순화하고, 서버 확장에 즉각 대응할 수 있는 유연한 CI/CD 아키텍처를 완성

끝.